

Métodos de Ordenamiento

Operaciones con Arreglos

- **Ordenar un arreglo**

Es la operación de ordenar un arreglo en algún orden de acuerdo a un criterio de ordenamiento.

Ordenamientos internos: Son aquellos en los que los valores a ordenar están en memoria principal, por lo que se asume que el tiempo que se requiere para acceder cualquier elemento sea el mismo (a [1], a [500], etc.).

Ordenamientos externos: Son aquellos en los que los valores a ordenar están en memoria secundaria (disco, cinta, cilindro magnético, etc.), por lo que se asume que el tiempo que se requiere para acceder a cualquier elemento depende de la última posición accesada (posición 1, posición 500, etc.).

Algoritmos de ordenamiento.

• Internos:

- 1. Inserción (InsertionSort).**
- 2. Selección (SelectionSort).**
- 3. Intercambio.**
- 4. Ordenamiento de árbol.**
- 5. QuickSort.**
- 6. MergeSort.**
- 7. RadixSort.**

Externos:

- 1. Straight merging.**
- 2. Natural merging.**
- 3. Balanced multiway merging.**
- 4. Polyphase sort.**
- 5. Distribution of initial runs.**

Burbuja (Bubble Sort): Compara elementos adyacentes y los intercambia si están desordenados. Es sencillo pero ineficiente para conjuntos grandes.

Inserción (Insertion Sort): Toma un elemento desordenado y lo inserta en la posición correcta dentro de una sublista ya ordenada.

Selección (Selection Sort): Busca el elemento más pequeño y lo coloca al inicio, repitiendo el proceso para el resto de la lista.

Rápido (Quicksort): Utiliza un elemento "pivote" para dividir la lista en dos subconjuntos (menores y mayores al pivote) y los ordena recursivamente.

Combinación (Merge Sort): Divide la lista en subconjuntos más pequeños, los ordena y luego los fusiona.

Algoritmos de intercambio:

En este tipo de algoritmos se toman los elementos de dos en dos, se comparan y se INTERCAMBIAN si no están en el orden adecuado. Este proceso se repite hasta que se ha analizado todo el conjunto de elementos y ya no hay intercambios.

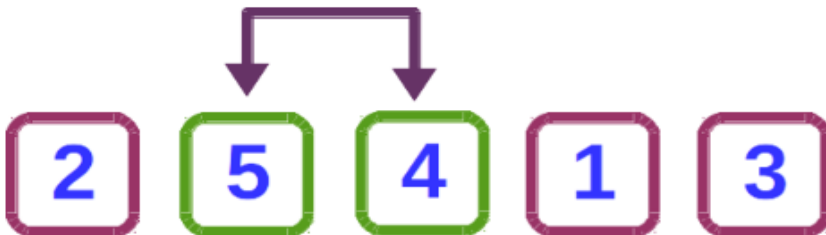
Entre estos algoritmos se encuentran el Burbuja (BubbleSort) y QuickSort

5 2 4 1 3

Tomamos los primeros dos números y si no están ordenados se intercambian los lugares



Se repite el proceso con los siguientes dos números



El proceso continua hasta llegar al final



En la tercera iteración no se evalúan los últimos dos valores



La cuarta iteración se finaliza sin que se haya realizado un intercambio por lo que el algoritmo termina



Al finalizar el algoritmo tenemos como resultado la lista ordenado



Operaciones de Ordenamiento

Algoritmos de inserción:

En este tipo de algoritmo los elementos que van a ser ordenados son considerados uno a la vez.

Su funcionamiento consiste en el recorrido por la lista seleccionando en cada iteración un valor como clave y compararlo con el resto insertándolo en el lugar correspondiente.

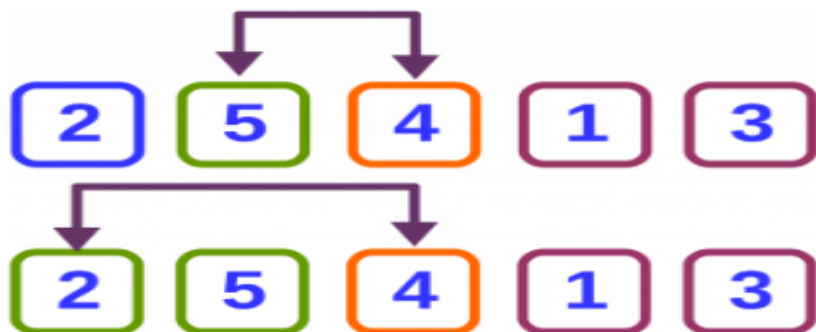
Entre estos algoritmos se encuentran el de Inserción Directa, ShellSort, Inserción Binaria y Hashing.



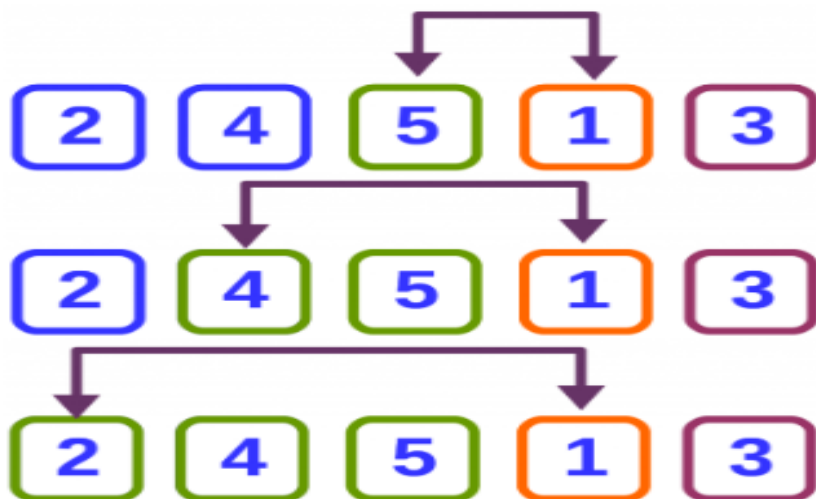
Se selecciona el segundo valor como clave y se lo compara con los valores ubicados a su izquierda. Si el valor es menor entonces se inserta en el lugar correspondiente.



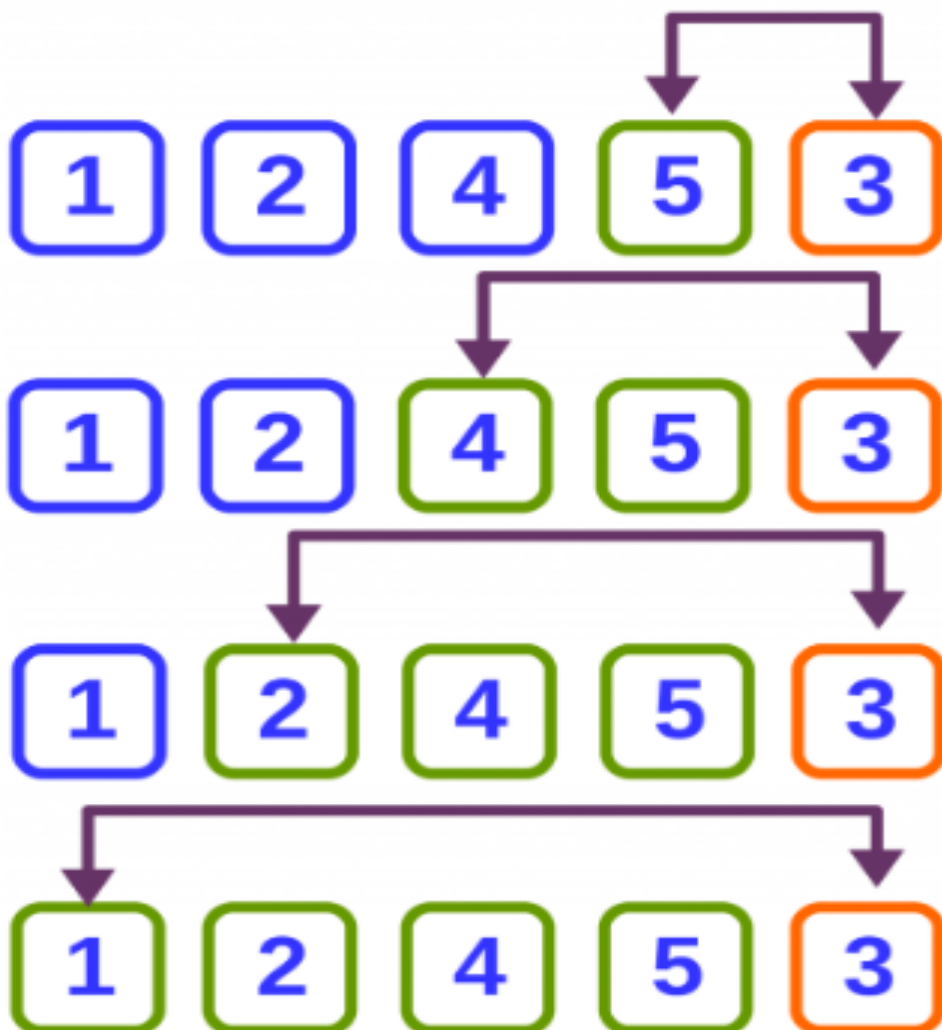
Se selecciona el siguiente número como clave y se repite el proceso para todos los valores anteriores. En el siguiente caso la clave 4 se compara primero con 5 y luego con 2. Al ser menor que el primer caso comparado y mayor que el segundo se lo inserta entre ambos números.



Se selecciona la siguiente clave. Se sigue comparando con cada número a su izquierda hasta encontrar uno que sea menor o llegar al principio de la lista.



Finalmente se selecciona la última clave.



Al finalizar el algoritmo tenemos como resultado la lista ordenada.



Algoritmos de selección:

En este tipo de algoritmos se SELECCIONA o se busca el elemento más pequeño (o más grande) de todo el conjunto de elementos y se coloca en su posición adecuada. Este proceso se repite para el resto de los elementos hasta que todos son analizados.

Entre estos algoritmos se encuentra el de Selección Directa.

Su funcionamiento es sencillo, tenemos la siguiente lista desordenada:

```
lista = [3, 1, 4, 2]
```

Primero buscamos al mínimo elemento de la lista (el 1). Luego lo intercambiamos con el primer elemento (el 3). Y obtendrás:

```
lista = [1, 3, 4, 2]
```

Segundo, otra vez buscamos al mínimo elemento de la lista, pero esta vez ignorando al primer elemento porque está ordenado. El mínimo es el 2. Intercambiamos con el segundo elemento (el 3). Obtienes:

```
lista = [1, 2, 4, 3]
```

Tercero, buscamos al menor de la lista, ignorando al primero y segundo elemento porque están ordenados. El mínimo es el 3. Intercambiamos con el tercer elemento (el 4). Obtienes la lista ordenada:

```
lista = [1, 2, 3, 4]
```

Así de sencillo funciona este algoritmo...